

Análise de Valor no Mercado Automóvel

Desenvolvimento de um Sistema Integrado Local e Web com Machine Learning para Avaliação de Carros Usados

Juliana Rodrigues (1231182)

Geovani Junior (1131518)

Instituto Superior de Engenharia do Porto

Versão Final — Junho de 2026

Resumo

O presente trabalho descreve o desenvolvimento de um sistema integrado para a recolha automática, a análise de dados e a comercialização de anúncios de veículos usados provenientes do mercado online OLX Portugal. A solução assenta numa arquitetura de componentes integrados: (1) um scraper em Python (`olx_car_scraper.py`) que recolhe dados estruturados de anúncios de automóveis, incluindo marca, modelo, ano, quilometragem, preço, potência e localização; (2) um módulo de inteligência artificial (`gerar_previsoes_ia.py`) que utiliza algoritmos de aprendizagem automática, nomeadamente XGBoost, Random Forest, LightGBM e modelos de regressão linear/exponencial, para estimar o valor justo de mercado de cada veículo com base nas suas características; (3) um workflow de automação construído na plataforma n8n, responsável pela filtragem dos anúncios já avaliados e pelo envio de alertas via Telegram sempre que surge um veículo abaixo do valor estimado; (4) um portal web de comercialização desenvolvido em HTML5, CSS3 e JavaScript vanilla, com filtros avançados e indicadores visuais de oportunidade; e (5) uma aplicação desktop desenvolvida em C++ com o framework Qt (`GeOLXavani`) que permite a gestão local do pipeline, treino de modelos e avaliação personalizada através de scripts auxiliares. O sistema evidencia o potencial da unificação de dados e modelos preditivos persistidos em formato padrão para mitigar a assimetria de informação no mercado automóvel nacional.

Keywords: Web Scraping, Machine Learning, XGBoost, Random Forest, LightGBM, n8n, Qt C++, Mercado Automóvel, OLX, Portal Web, Python, Data Science.

1 Introdução

O mercado de automóveis usados em Portugal movimenta anualmente centenas de milhares de transações. Plataformas como o OLX Portugal concentram uma parte significativa deste volume, com milhares de novos anúncios publicados diariamente. A elevada volatilidade dos preços, a depreciação não linear dos veículos e o elevado número de listagens tornam difícil, para um comprador individual, identificar manualmente as oportunidades com melhor relação qualidade/preço.

Visando mitigar a assimetria de informação existente, este projeto propõe o desenvolvimento de uma arquitetura unificada, baseada em automação, Inteligência Artificial e interfaces local e web. A primeira camada é responsável pela extração sistemática de dados brutos do mercado. A segunda foca-se no pré-processamento de dados e no treino de modelos preditivos para estimar o valor comercial justo de cada automóvel. Por sua vez, a terceira camada orquestra a análise destas avaliações, emitindo alertas em tempo real perante oportunidades de negócio vantajosas. Por fim, o ecossistema disponibiliza interfaces duplas: um portal Web interativo, que centraliza esta inteligência analítica para apoiar a decisão final do consumidor, e uma aplicação desktop nativa desenvolvida em C++/Qt que

permite o controlo do scraper, retreino de modelos e uma ferramenta de avaliação manual de veículos offline baseada em modelos persistidos.

2 Arquitetura do Sistema

O sistema é composto por módulos interdependentes que formam um pipeline de dados completo. O fluxo de execução é orquestrado de forma a garantir a passagem sequencial e íntegra dos dados entre a extração, o processamento preditivo, a interface visual e as ferramentas desktop.

2.1 Módulo de Recolha de Dados (`olx_car_scraper.py`)

O primeiro módulo do sistema é responsável pela aquisição automatizada de dados reais e atualizados sobre o mercado automóvel. Para o efeito, desenvolveu-se um scraper em Python direcionado à plataforma OLX. O componente opera em modo headless por omissão, mimetiza o comportamento de um utilizador humano através de pausas aleatórias entre pedidos (1,8–4,2 segundos) e rotação de user-agents, e aceita automaticamente os banners de consentimento de cookies. Durante o parsing dos dados detalhados, o scraper extrai também atributos específicos de hardware, tais como a potência em cavalos (`potencia_cv`) a partir dos campos do anúncio.

2.2 Filtros de Ano e Preço

A função `passa_filtros()` implementa lógica de filtragem em dois momentos: após o parsing do cartão de listagem (filtragem rápida, evita visitas desnecessárias à página de detalhe) e após a visita ao detalhe (filtragem final com dados completos). A função aceita limites mínimos e máximos para ano e preço, tratando valores `None` como “sem restrição” para não descartar anúncios onde o campo ainda não foi extraído.

3 Módulo de Previsão de Preços por IA (`gerar_previsoes_ia.py`)

Este módulo atua como o motor analítico do projeto. O seu objetivo principal é processar os dados brutos recolhidos no portal OLX, treinar múltiplos algoritmos de Machine Learning em simultâneo e prever o valor justo de mercado para cada veículo. O resultado final enriquece a base de dados do front-end, fornecendo ao utilizador uma avaliação imediata sobre a qualidade do negócio. Para garantir a precisão e robustez do sistema, o desenvolvimento deste módulo foi dividido em três fases fundamentais: preparação de dados, modelagem preditiva e persistência física.

3.1 Preparação e Limpeza de Dados

Antes do treino da Inteligência Artificial, os dados extraídos pelo scraper passam por um rigoroso processo de limpeza. Esta fase inclui:

- **Tratamento de Anomalias:** Remoção de anúncios sem preço definido ou com valores fora dos limites plausíveis de mercado (ex. quilometragens irrealistas ou preços extremos).
- **Imputação de Dados Faltantes:** Quando atributos cruciais, como os quilómetros ou o ano de fabrico, não são detetados, o sistema utiliza a mediana global do conjunto de dados para preencher a lacuna.

- **Estimação de Potência por Heurísticas:** Caso a potência (**hp**) não tenha sido capturada no scraper, o sistema aplica um processamento de texto baseado em termos frequentes no título (como “320d”, “1.6 tdi”) e a marca do veículo para atribuir uma potência estimada razoável, eliminando a anterior limitação de atribuir um valor flat de 100 cv.
- **Engenharia de Features:** Conversão de variáveis categóricas (Marca, Modelo simplificado, Combustível e Transmissão) em codificação dummy (**get_dummies**) e criação de novas variáveis temporais como a idade do carro (**age**) e a média de quilômetros percorridos por ano (**km_per_year**). Para os modelos lineares, as features numéricas passam por uma normalização estatística (**StandardScaler**).

3.2 Modelos Utilizados

Foram avaliados seis modelos de regressão, cobrindo desde abordagens lineares simples até métodos de ensemble baseados em árvores de decisão. A descrição metodológica de cada modelo está detalhada na Tabela 1.

Tabela 1: Modelos de regressão avaliados.

Modelo	Descrição
Regressão Linear	Modelo de referência (baseline) com relação linear direta entre variáveis.
Regressão Exponencial	Aplica o logaritmo ao target (y), captando a desvalorização percentual.
Regressão Logarítmica	Aplica logaritmo às features numéricas, modelando o efeito de quilômetros.
Random Forest	Conjunto de árvores de decisão que captura não-linearidades do mercado.
XGBoost	Gradient boosting com regularização, focado na otimização de interações.
LightGBM	Gradient boosting otimizado para rapidez e árvores orientadas a folhas.

3.3 Métricas de Avaliação

A escolha do modelo final assenta principalmente no MAPE (Mean Absolute Percentage Error), em detrimento do MAE absoluto. Esta opção justifica-se pela natureza do dataset, que cobre veículos com preços muito díspares: um erro absoluto de, por exemplo, 1000 EUR representa um desvio relativo muito diferente consoante o veículo custe 5.000 EUR ou 50.000 EUR. O MAPE normaliza esta diferença, permitindo uma comparação mais interpretável entre modelos. As restantes métricas (MAE, RMSE, R^2) são reportadas como complemento.

A Tabela 2 apresenta as métricas resultantes obtidas na execução final sobre o conjunto de teste unificado de 501 anúncios reais.

Tabela 2: Métricas de avaliação obtidas por modelo.

Modelo	MAE (EUR)	RMSE (EUR)	R^2	MAPE (%)
Regressão Linear	10.237,2	17.813,4	0,612	108,9%
Regressão Exponencial	8.411,0	17.500,1	0,625	51,8%
Regressão Logarítmica	9.690,2	17.422,6	0,629	84,2%
Random Forest	11.172,2	21.908,3	0,413	115,5%
XGBoost	11.016,5	20.604,8	0,481	93,7%
LightGBM	10.963,3	21.003,5	0,460	114,5%

O modelo baseado em **Regressão Exponencial (Ridge)** apresentou o melhor compromisso global, registando o menor erro percentual médio (MAPE de 51,8%) e o menor

desvio absoluto médio (MAE de 8.411,0 EUR), sendo selecionado para a produção e persistido.

3.4 Gráficos Comparativos

O script exporta a figura de comparação de modelos (`comparacao_modelos.png`) contendo gráficos de dispersão (preço previsto vs real) e comparadores de barras para as métricas, permitindo diagnosticar o overfitting de algoritmos com elevada flexibilidade como o Random Forest no nosso dataset atual.

3.5 Do Terminal para Produção

A transição da fase exploratória para a fase de produção traduz-se no uso do script `gerar_previsoes_ia.py`, executado em modo batch sem interação do utilizador. Este processa o dataset unificado completo e grava as previsões resultantes no ficheiro `olx_carros_ia.json`, integrado no portal web através dos campos `preco_ia` e `status_ia`.

3.6 Integração com o Pipeline

Ao contrário de uma integração direta no workflow n8n, a chamada ao módulo de IA é feita pelo script unificador `executar_tudo.py`, que invoca sequencialmente o scraper e o módulo de previsão. O n8n atua após a conclusão deste pipeline, lendo o ficheiro já enriquecido e aplicando a lógica de deteção de oportunidades.

4 Workflow de Automação com n8n

O n8n opera como camada de orquestração assíncrona dos alertas: é executado de forma agendada, lê o ficheiro final e notifica o utilizador caso sejam identificados desvios favoráveis. O fluxo proposto é composto por nove nós funcionais, descritos na Tabela 3.

Tabela 3: Nós do workflow n8n e respetivas funções.

#	Nó	Função
1	Schedule Trigger	Execução periódica agendada (e.g. a cada 6 horas).
2	Execute Command	Executa o script <code>executar_tudo.py</code> para iniciar o pipeline.
3	Read/Write Files	Lê o ficheiro final gerado pelo previsor de IA.
4	Spreadsheet File	Converte o ficheiro em registos estruturados.
5	Filter / IF	Aplica filtros básicos de preço, ano e marca.
6	DB Lookup	Verifica se o anúncio já foi processado e armazenado.
7	DB Insert	Guarda novos anúncios na base de dados (Airtable/Supabase).
8	Code (JS)	Formata a mensagem de alerta com os dados detalhados do veículo.
9	Switch + Telegram	Envia a notificação com os desvios calculados.

4.1 Deteção de Oportunidades

O nó de formatação calcula o desvio de cada anúncio relativamente ao preço de referência, que corresponde à previsão da IA (`preco_ia`) quando disponível, ou à mediana histórica como fallback. Anúncios com preço inferior à referência em mais de 10% são classificados como “oportunidade” e desencadeiam o envio do alerta via Telegram.

5 Portal Web de Comercialização

O portal AutoMercado é uma aplicação web de página única (SPA) desenvolvida em HTML5, CSS3 e JavaScript vanilla. A interface oferece uma barra de pesquisa rápida, filtros avançados (preço, ano, quilometragem, combustível e transmissão), ordenação dinâmica e um sistema de favoritos com persistência local.

5.1 Indicadores de Preço

Cada card de veículo exibe o desvio em relação ao valor justo estimado pela IA. O desvio é realçado visualmente a verde (com a poupança associada) se o valor pedido estiver abaixo do previsto pela IA, ou a vermelho caso contrário, orientando intuitivamente as decisões de compra.

6 Aplicação Desktop Qt (GeOLXavani)

Para além do portal web, o ecossistema do projeto foi expandido para incluir uma aplicação desktop desenvolvida em C++ com o framework Qt (Qt Widgets). Esta aplicação (GeOLXavani) destina-se a utilizadores que necessitam de uma gestão local mais robusta, oferecendo controlos diretos sobre a recolha de dados, treino de modelos e avaliação de veículos individualmente.

A interface está estruturada em quatro widgets principais interligados através de um controlador central (`MainWindow`):

- **ScraperWidget:** Permite controlar a execução do script Python de web scraping através de uma interface visual. O utilizador pode configurar parâmetros como o número de páginas a extrair e ver em tempo real o log de execução do terminal diretamente numa caixa de texto.
- **MLWidget:** Oferece controlos para iniciar o treino local dos modelos de regressão a partir da interface gráfica, lendo o ficheiro de dados unificado e reportando as métricas de performance.
- **TrainingWidget:** Exibe as métricas comparativas resultantes do treino dos modelos e permite despoletar a geração de previsões em lote (`gerar_previsoes_ia.py`), atualizando o ficheiro JSON e gravando o modelo preditivo final.
- **CarSearchWidget:** Apresenta uma tabela interativa com todos os veículos registados no ficheiro unificado. Possui opções de filtragem e pesquisa por marca ou modelo, e integra um botão para avaliação personalizada do veículo.

A avaliação personalizada é executada no widget `CarEvaluationWidget`, o qual invoca o script independente `evaluate_car.py`. Este script carrega o ficheiro de modelo guardado (`modelo_ia.pkl`) e calcula o preço estimado para as características exatas introduzidas pelo utilizador (marca, modelo, quilometragem, potência, ano, combustível e tipo de transmissão), devolvendo o resultado para exibição imediata no ecrã.

A unificação dos dados é um pilar fundamental da arquitetura: todos os caminhos de leitura e escrita nas classes C++ e scripts de suporte foram alterados para apontar diretamente para os ficheiros centralizados localizados na pasta `JuOLXana` (`olx_carros.csv`, `olx_carros.json` e `olx_carros_ia.json`). Isto garante que a aplicação desktop e o portal web operam sob a mesma base de dados.

7 Considerações Técnicas e Limitações

A principal limitação do sistema reside na fragilidade inerente ao web scraping: alterações ao HTML do OLX (seletores CSS, estrutura do DOM, mecanismos de proteção anti-bot) podem inutilizar os parsers sem aviso prévio. Do ponto de vista ético e legal, o sistema opera em modo de leitura e não simula transações nem acede a áreas autenticadas. O ritmo de pedidos respeita os limites razoáveis de cortesia para com o servidor de origem.

7.1 Melhorias Efetuadas e Resolução de Limitações

O desenvolvimento final do sistema permitiu mitigar por completo as três principais limitações identificadas na fase exploratória da Inteligência Artificial:

1. **Ausência da variável potência:** O scraper de recolha de dados foi atualizado para extrair dinamicamente a potência (`potencia_cv`) diretamente da página de detalhes de cada anúncio. Adicionalmente, implementou-se uma heurística de estimação de potência com base em termos frequentes do título (ex. “320d”, “1.6 tdi”) e médias de marca/modelo para preencher valores em falta. A inclusão da potência como feature activa resultou num aumento drástico do R^2 de ~ 0.35 para mais de 0.60 nos principais modelos regressivos.
2. **Falta de persistência do modelo:** Criou-se um mecanismo de gravação que exporta o estado completo do melhor modelo selecionado para o ficheiro `modelo_ia.pkl`. O estado persistido inclui os pesos do regressor, o mapeamento de variáveis categóricas frequentes e a normalização de features numéricas (`StandardScaler`).
3. **Problemas de importação de classes na deserialização:** Para evitar falhas ao importar classes customizadas entre diferentes pastas (`GeOLXavani` e `JuOLXana`), o ficheiro pickle passou a armazenar apenas estruturas nativas de Python e do `scikit-learn/xgboost`. O script de avaliação (`evaluate_car.py`) reconstrói a matriz de features e as dummy variables dinamicamente a partir deste dicionário de estado, garantindo total desacoplamento e estabilidade.

8 Conclusão

Este trabalho demonstrou o desenvolvimento de um produto de dados completo, do princípio ao fim. Foi comprovado que, com ferramentas de código aberto (Python, `scikit-learn`, `n8n`) e de desenvolvimento nativo (Qt C++), é possível transcender a simples extração de informação, incorporando inteligência analítica baseada em Machine Learning para resolver assimetrias de informação no mercado automóvel. A orquestração do scraper com os algoritmos preditivos fornece métricas acionáveis, prontamente comunicadas ao utilizador através do portal web, aplicação desktop e das notificações automáticas.

Como trabalho futuro, identifica-se a integração de modelos de Natural Language Processing (NLP) para análise semântica das descrições dos vendedores, permitindo a deteção automática de sinistros ou extras de luxo; a integração com outras fontes de dados (`Standvirtual`, `CustoJusto`); e a migração para uma stack com base de dados persistente (`PostgreSQL` + `API REST`) que permita armazenamento histórico, multi-utilizador e otimização contínua dos modelos preditivos.

Referências

- [1] Richardson L (2007) Beautiful Soup Documentation. Crummy.com. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

- [2] Selenium Project (2024) Selenium WebDriver Documentation. <https://www.selenium.dev/documentation/>
- [3] n8n GmbH (2024) n8n Documentation — Workflow Automation Tool. <https://docs.n8n.io/>
- [4] OLX Portugal (2025) Carros, Motos e Barcos — Anúncios. <https://www.olx.pt/carros-motos-e-barcos/carros/>
- [5] McKinney W (2017) Python for Data Analysis, 2nd edn. O'Reilly Media, Sebastopol.
- [6] Chen T, Guestrin C (2016) XGBoost: A Scalable Tree Boosting System. KDD '16. <https://doi.org/10.1145/2939672.2939785>
- [7] Ke G et al. (2017) LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.